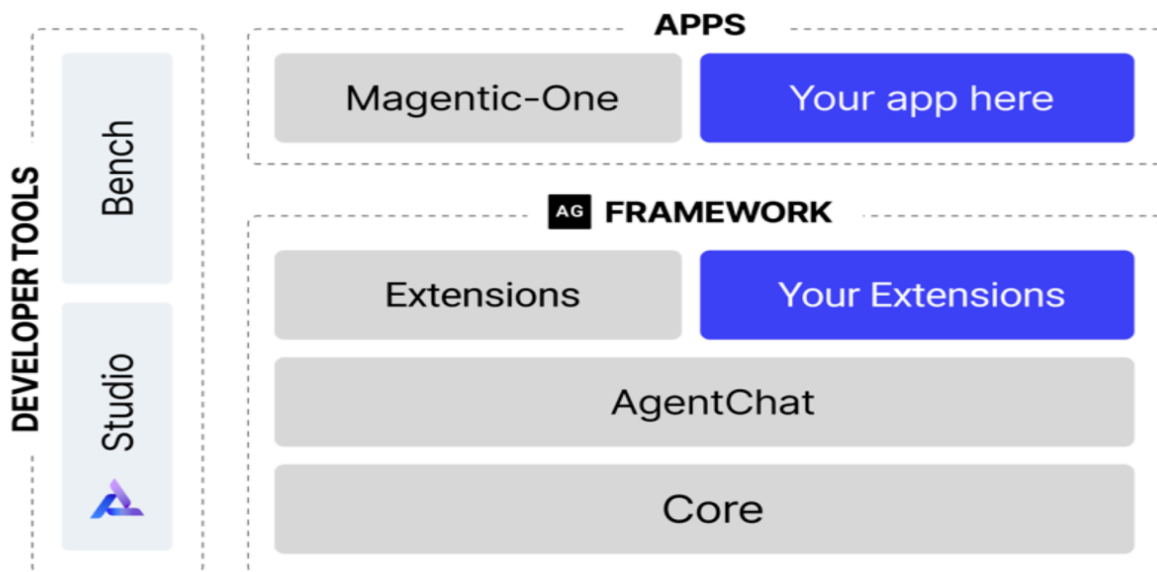


Autogen

AutoGen v0.4 is a framework for building multi-agent AI systems with an asynchronous, event-driven architecture. Key improvements over previous versions of autogen (current version is 0.4) include:

- A redesigned Core layer for scalability.
- AgentChat, a simplified high-level API for agent interactions.
- Extensions for integrating external tools and data.

Autogen Architecture

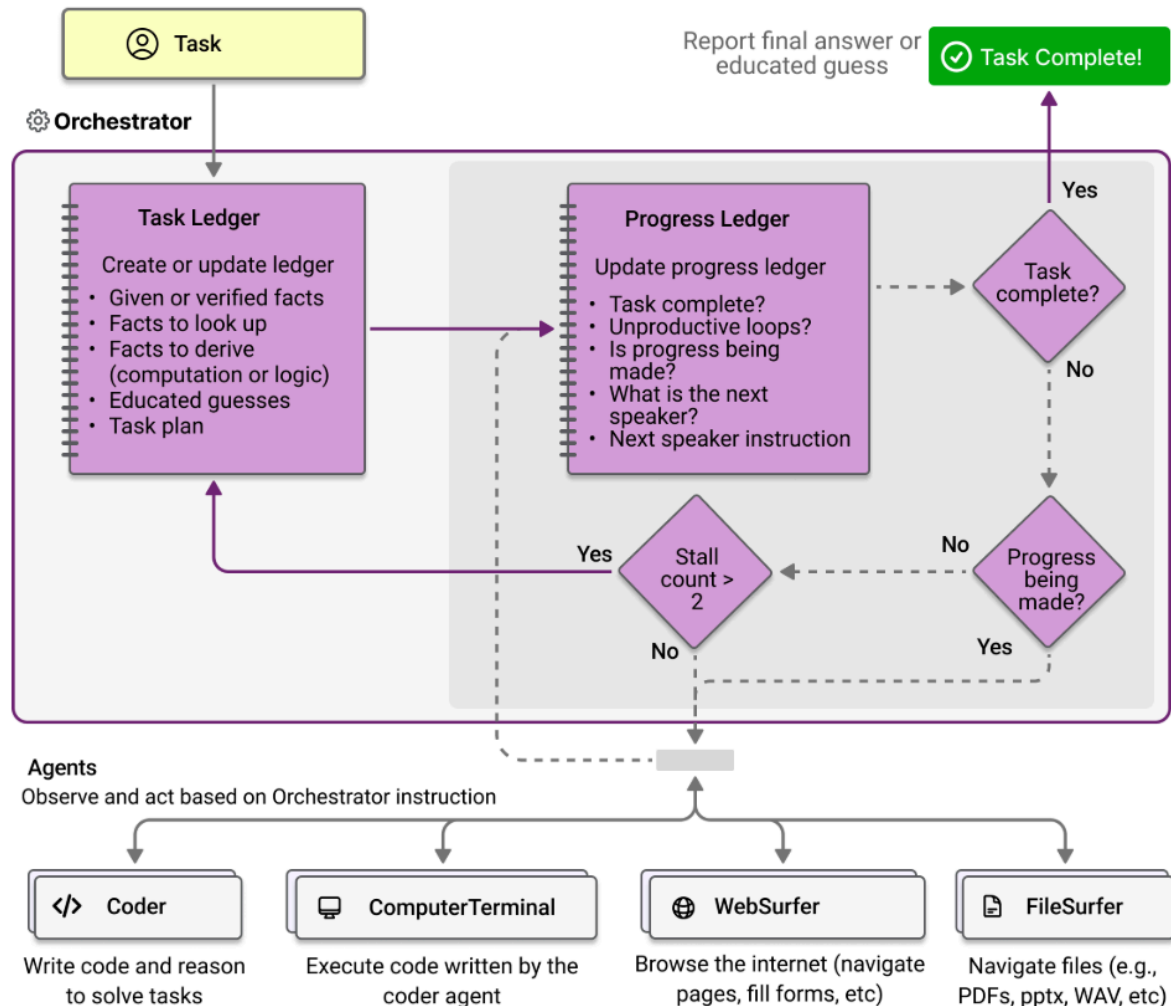


The architecture is divided into three main sections:

- **App's:** Pre-built and custom applications.
- **AG Framework:** Core components powering agents.
- **Developer Tools:** Tools for testing and building.

1) App's Layer

a. Magentic-One



- Purpose:

> A pre-built application demonstrating multi-agent collaboration (e.g., group problem-solving).

> Magentic-One employs a multi-agent architecture where a lead agent, the Orchestrator, directs four other agents to solve tasks. The Orchestrator plans, tracks progress, and re-plans to recover from errors, while directing specialized agents to perform tasks like operating a web browser, navigating local files, or writing and executing Python code.

> The Orchestrator begins by creating a plan to tackle the task, gathering needed facts and educated guesses in a *Task Ledger* that is maintained. At each step of its plan, the Orchestrator creates a *Progress Ledger* where it self-reflects on task progress and checks whether the task is completed. If the task is not yet completed, it assigns one of Magentic-One other agents a subtask to complete. After the assigned agent completes its subtask, the Orchestrator updates the *Progress Ledger* and continues in this way until the task is complete. If the Orchestrator finds that progress is not being made for enough steps, it can update the *Task Ledger* and create a new plan. This is illustrated in the figure above; the Orchestrator work is thus divided into an outer loop where it updates the *Task Ledger* and an inner loop to update the *Progress Ledger*.

> **Magentic-One consists of the following agents:**

- **Orchestrator:** The lead agent responsible for task decomposition, planning, directing other agents in executing subtasks, tracking overall progress, and taking corrective actions as needed
- **WebSurfer:** An LLM-based agent proficient in commanding and managing the state of a Chromium-based web browser. For each request, the WebSurfer performs actions such as navigation (e.g., visiting URLs, performing searches), interacting with webpages (e.g., clicking, typing), and reading actions (e.g., summarizing, answering questions). It then reports on the new state of the webpage. The WebSurfer relies on the browser's accessibility tree and set-of-marks prompting it to perform its tasks.
- **FileSurfer:** An LLM-based agent that commands a markdown-based file preview application to read local files. It can also perform common navigation tasks such as listing directory contents and navigating through them.
Note- Imagine you have a text file (like notes.md), and you want to see its contents nicely formatted (with headings, bold text, lists, etc.) instead of plain text, A markdown-based file preview application is a tool that does this.
- **Coder:** An LLM-based agent specialized in writing code, analyzing information collected from the other agents, and creating new artifacts.
- **ComputerTerminal:** Provides access to a console shell for executing programs and installing new libraries.

> **Evaluation-** To rigorously evaluate Magentic-One's performance, we introduce AutoGenBench, an open-source standalone tool for running agentic benchmarks that allows repetition and isolation, e.g., to control for variance of stochastic LLM calls and side-effects of agents taking actions in the world.

- **Role:** Serves as a reference implementation for developers.
- **GitHub Reference:** Explore the code at [\[microsoft/autogen\]\(https://github.com/microsoft/autogen/tree/main/notebook\)](https://github.com/microsoft/autogen/tree/main/notebook).

b. Your App Here

- **Purpose:** A customizable space for user-defined applications.
- **Use Cases:**
 - Customer service chatbots.
 - AI research assistants.
 - Automated content generation workflows.
- **Flexibility:** Integrates with LLMs (GPT-4, local models) via the AG Framework.

2. AG Framework Layer

a. Core

- This is the older version which we are not using currently, now we are using the updated version 'AgentChat' which is below.
- 'Core' is the foundational building blocks for an event-driven agentic system.
- **Design:** Asynchronous, event-driven architecture using an actor model.
- **Key Features:**
 - Agents act independently and react to events (e.g., messages, API calls).
 - Supports parallel task execution for scalability.

b. AgentChat

- **Purpose:** Simplifies building conversational agents.

> AgentChat is a task-driven, high-level API built on the core layer, featuring group chat, code execution, pre-built agents, and more.

> AgentChat is a high-level API for building multi-agent applications. It is built on top of the autogen-core package. For beginner users, AgentChat is the recommended starting point. For advanced users, autogen-core's event-driven programming model provides more flexibility and control over the underlying components.

- **Key Features:**
 - Predefined agent roles (e.g., 'AssistantAgent', 'UserProxyAgent').
 - Handles message routing and state management.

c. Extensions

- **Purpose:** Extend functionality via plugins.

> Extensions are implementations of core interfaces and third-party integrations, such as the Azure code executor and OpenAI model client.

Eg: autogen is providing the extension for LLM models (check the 'Extension' section in its doc)

- **Types:**

- Official Extensions: OpenAI/Azure connectors, math tools.
- Custom Extensions: Integrate APIs (e.g., weather data, databases).

3. Developer Tools

a. Bench

- **Purpose:** Benchmark agent performance and scalability.

> It enables developers to benchmark their agents by measuring and comparing performance across tasks and environments.

- **Use Cases:**

- Measure latency under heavy loads.
- Optimize resource usage (e.g., API costs).

- **Documentation:** [Performance

Testing](<https://microsoft.github.io/autogen/stable/user-guide/performance.html>).

b. Studio

- **Purpose:** No-code/low-code agent design.

> AutoGen Studio is a low-code interface built to help you rapidly prototype AI agents, enhance them with tools, compose them into teams and interact with them to accomplish tasks. It is built on AutoGen AgentChat - a high-level API for building multi-agent applications.

Capabilities - What Can You Do with AutoGen Studio?

1) Team Builder

* A visual interface for creating agent teams through declarative specification (JSON) or drag-and-drop

- * Supports configuration of all core components: teams, agents, tools, models, and termination conditions
- * Fully compatible with AgentChat's component definitions.

2) Playground

*Interactive environment for testing and running agent teams

*Features include:

- Live message streaming between agents
- Visual representation of message flow through a control transition graph
- Full run control with the ability to pause or stop execution.

3) Gallery: Enables easy integration of third-party components

4) Deployment: Export and run teams in python code

- Features:

- Drag-and-drop workflow builder.
- Interactive sandbox for testing.
- Visual debugging tools.

Types of Agents in Autogen

1)UserProxyAgent

Imagine you're playing a game where you (a human) are chatting with an AI assistant. But instead of typing directly, you have a robot helper that:

- Listens to the AI's questions
- Asks you for input (like showing a pop-up on your screen)
- Passes your reply back to the AI

This robot helper is the UserProxyAgent. Here the humans have to manually reply—the UserProxyAgent is just a "middleman"

Key Things to Remember

✅ **Blocking Mode:** The AI waits until you reply (like a phone call on hold).

🕒 **Timeouts Matter:** If you don't reply in time, it should cancel (like a call dropping after 30 seconds).

🔄 **Works with Apps:** Can connect to websites (FastAPI), chatbots (ChainLit), or even console input.

Example (Like a Pizza Order Bot)

```
async def pizza_order():
    agent = UserProxyAgent("pizza_bot_helper")
    reply = await agent.ask_user("What pizza do you want?")
    print(f"You ordered: {reply}")
```

2) AssistantAgent

The AssistantAgent is a specialized chat agent designed to provide assistance with the ability to use tools. Here's a detailed breakdown of its components and functionality:

Core Characteristics

Inheritance: Inherits from BaseChatAgent and Component[AssistantAgentConfig]

Purpose: Provides an AI assistant that can use tools to complete tasks

Termination: Can signal task completion by replying with "TERMINATE"

Core Functionality

Message Handling:

on_messages(): Processes messages and returns a final Response

on_messages_stream(): Async generator that streams intermediate messages and ends with final Response

Task Execution:

run(): Returns a TaskResult containing all produced messages (last is final response)

run_stream(): Async generator version that streams messages and ends with TaskResult

Key Parameters

- `name`: Identifier for the agent
- `model_client`: The chat completion client used for AI interactions
- `tools`: List of available tools (can be `BaseTool` instances or callable functions)
- `workbench`: Workspace environment for the agent
- `handoffs`: List of handoff protocols or identifiers
- `model_context`: Context for chat completions
- `description`: Brief description of the agent's purpose

- `system_message`: Initial instruction for the AI (default emphasizes tool use and TERMINATE)
- `model_client_stream`: Whether to use streaming with the model client

3) CodeExecutorAgent

The CodeExecutorAgent is like a smart coding assistant that can both generate and run code for you. Let me break it down in simple terms:

What It Does

- Code Generation & Execution: It can write code based on your instructions and then run that code to see the results.
- Safe Execution: It runs code in isolated environments (like Docker containers) so your main computer stays safe.
- Team Player: It can work alone or with other AI agents in a group.

Main Components

- Name: Just an identifier for the agent (like "PythonHelper")
- Code Executor: The engine that actually runs the code (recommended to use Docker for safety)
- Model Client: Optional AI brain that generates the code (like GPT-4)
- Settings: Various options to control how it behaves

How You Can Use It

Option 1: With Code Generation (using model_client)

- You ask "Write Python code to print hello world"
- The agent:
 1. Generates the code using its AI model
 2. Runs the code in a safe container
 3. Shows you both the code and the output
 4. Can even explain if something went wrong

Option 2: Just Code Execution (without model_client)

- You send it code blocks (marked with `python` or `sh`)
- It simply runs that code and shows you the output
- Useful when another AI agent generates the code

Selector Group Chat

SelectorGroupChat is a type of team in AutoGen that lets multiple agents talk to each other. But unlike round-robin (where agents talk one after another in a fixed order), **SelectorGroupChat uses a model (like GPT) to decide which agent should talk next.**

Why use it?

In real-world teamwork:

- Not everyone needs to speak in order.
- The right person should talk depending on the situation.

That's what SelectorGroupChat does — it dynamically picks **who should respond** next, based on the **current task and previous messages**.

Components Explained

1. Agents

You define a list of agents. Each has:

- A **name**
- A **description** (what their role is, like "I search the web" or "I do calculations")
- Optionally: tools they can use.

2. SelectorGroupChat

This is the team manager:

- It selects one agent at a time to respond.
- The selection is done using an **LLM (e.g., GPT-4)**.
- It checks the history, the descriptions of agents, and then picks who should speak next.

3. Conversation History

It keeps track of the entire conversation (unless you manually reset it). The model uses this history to make smart choices.

How It Works (Step-by-Step)

Step 1: Setup Agents

You create agents with roles like:

```
planning_agent = AssistantAgent(name="Planner", description="Plans out the task")
websearch_agent = AssistantAgent(name="Searcher", description="Searches the web")
data_analyst = AssistantAgent(name="Analyst", description="Does math and analysis")
```

Step 2: Setup the SelectorGroupChat

```
python Copy Edit

team = SelectorGroupChat(
    members=[planning_agent, websearch_agent, data_analyst],
    allow_repeated_speaker=False # default is False
)
```

Step 3: Run a Task

```
python Copy Edit

result = team.run("Analyze performance of Miami Heat team using recent stats.")
```



What Happens Internally:

1. First, the **task** is broadcasted to all agents.
 2. Then the **LLM picks** the most suitable agent to respond.
 3. That agent's response is then **shared with everyone**.
 4. Then again, the model picks **who speaks next**.
 5. It continues until a **termination condition** is met.
-



Termination Conditions

SelectorGroupChat ends when:

- A special keyword like "TERMINATE" is mentioned.
- A certain **number of messages** is reached (e.g., 25).
- You can combine these:

```
termination = TextMentionTermination("TERMINATE") | MaxMessageTermination(25)
```



Key Behaviors and Options



allow_repeated_speaker

- If False (default): same agent cannot talk twice in a row.
- If True: allows the same agent to speak again if it makes sense.
- This helps avoid getting stuck when only one agent is suitable to talk.



Custom Selector Prompt

You can customize the prompt the LLM uses to decide who should speak next.

🧩 Custom Selector Logic

You can also write your own function that filters candidates or directly picks the next agent.

```
def my_selector_fn(message, agents, history):  
    # custom logic to pick an agent  
    return some_agent
```

✅ Why It's Better Than RoundRobinGroupChat

Feature	SelectorGroupChat	RoundRobinGroupChat
Turn-taking	Smart selection by LLM	Fixed order
Relevance	High (context-aware)	May be irrelevant
Flexibility	Very flexible	Rigid
Ideal for	Complex tasks, many roles	Simple pipelines